

Analisis de Secuencias K-mers

Josue Samuel Philco Puma

21 de mayo de 2026

1. Introducción

El análisis de secuencias biológicas es una de las áreas fundamentales dentro de la bioinformática, ya que permite estudiar similitudes, diferencias y patrones presentes en el material genético de distintos organismos. Entre las técnicas más utilizadas para este tipo de análisis se encuentra el uso de k-mers, los cuales representan subsecuencias de longitud fija extraídas de una secuencia de ADN o ARN. El estudio de k-mers permite identificar regiones repetidas, patrones conservados y relaciones genómicas entre organismos.

En este trabajo se desarrollará una herramienta en C++ para el procesamiento y análisis de secuencias biológicas en formato FASTA. El sistema permitirá realizar el conteo de k-mers, calcular sus frecuencias de aparición y obtener estadísticas relevantes sobre la composición genómica de las secuencias analizadas. Además, se implementarán mecanismos de comparación entre organismos relacionados utilizando técnicas alignment-free basadas en conjuntos de k-mers e índices de similitud.

2. Ejercicios

2.1. Implementación de k-mers

Primeramente, debemos saber que el **k-mers** son subsecuencias de una longitud k que van a ser extraídas de una secuencia biológica y este va a servir para estudios de información genómica en la identificación de patrones biológicos relevantes. Para poder calcular el total de **k-mers** que se puede extraer de una secuencia n es mediante $n - k + 1$. Ahora que tenemos esta información vamos a ver que pasos necesitamos para poder calcular la cantidad de **k-mers**.

1. Debemos poder hacer la lectura de los archivos FASTA correspondiente, esto debemos hacerlo con una estructura de datos eficiente que sera un vector.
2. Las secuencias FASTA tienen saltos de línea, por lo que debemos procesarlas como una sola línea.
3. Con la secuencia ya leída se debe tener la generación de k-mers para extraer las subsecuencias de tamaño k , este valor puede ser distinto en cada uso.
4. Para poder contar la cantidad de k-mers se debe utilizar estructuras de datos eficientes como tablas hash.
5. Ordenar los k-mers de mayor a menor frecuencia para identificar patrones repetitivos, regiones conservadas y subsecuencias relevantes dentro del genoma.
6. Se utiliza paralelismo con OpenMP para que el procesamiento sea más grande para las secuencias largas haciendo uso de los hilos para su ejecución.
7. Todos los resultados obtenidos serán almacenados en archivos de texto y también en csv para que sean utilizados en gráficos más adelante.

Ahora que tenemos los pasos definidos vamos ver la primera parte de la lectura de los archivos FASTA, este paso ya es conocido por los trabajos anteriores.

```
1 struct FastaSequence {  
2     string header;  
3     string sequence;  
4 };  
5  
6 vector<FastaSequence> readFastaFile(const string& filename) {  
7     ifstream file_fasta(filename);  
8     vector<FastaSequence> sequences;
```

```

9
10 if (!file_fasta.is_open()) {
11     cerr << "Error opening file: " << filename << endl;
12     return sequences;
13 }
14
15 string line;
16 FastaSequence current_sequence;
17
18 while (getline(file_fasta, line)) {
19     if (line.empty()) continue;
20
21     if (line[0] == '>') {
22         if (!current_sequence.header.empty()) {
23             sequences.push_back(current_sequence);
24             current_sequence = FastaSequence();
25         }
26         current_sequence.header = line.substr(1); // Remove '>'
27     }
28     else {
29         current_sequence.sequence += line;
30     }
31 }
32
33 if (!current_sequence.header.empty()) {
34     sequences.push_back(current_sequence);
35 }
36
37 return sequences;
38 }

```

Ahora que tenemos lectura de los archivos FASTA pasamos con la implementación del contador de **K-mers**, como se dijo anteriormente, este cuenta con paralelización. Utilizamos **OpenMP** para paralelizar este procedimiento es para que el procesamiento que va a realizar va a involucrar una gran cantidad de operaciones donde algunas pueden volverse repetitivas, además que el algoritmo va a estar recorriendo toda la secuencia y extrayendo las subsecuencias de longitud k en cada posición.

```

1 #include <omp.h>
2 using KmerMap = unordered_map<string, int>;
3
4 KmerMap countMers(string sequence, int k) {
5     int n_threads = omp_get_max_threads();
6     vector<KmerMap> local_kmer_maps(n_threads);
7
8     #pragma omp parallel
9     {
10         int tid = omp_get_thread_num();
11         auto& local_map = local_kmer_maps[tid];
12
13         local_map.reserve(100000);
14
15         #pragma omp for schedule(dynamic, 10000)
16         for (size_t i = 0; i <= sequence.size() - k; ++i) {
17             string kmer = sequence.substr(i, k);
18             local_map[kmer]++;
19         }
20     }
21
22     KmerMap global;
23     for (auto& local : local_kmer_maps) {
24         for (const auto& [kmer, freq] : local) {
25             global[kmer] += freq;
26         }
27     }

```

```
28
29     return global;
30 }
```

Entonces, vamos a ejecutar esta función sobre dos genomas propuestos en este laboratorio que son *Salmonella enterica subsp. enterica serovar Typhimurium str. LT2* y *Escherichia coli str. K-12 substr. MG1655*. Además, lo podemos probar con valores distintos de k para ver la diferencia en los resultados.

Con $k = 6$

```
usuario@usuario-L00-15IRH8:~/K-mears$ ./Kmears sequence2.fasta 6
Sequence length: 4639675
Using k = 6

===== TOP 10 K-MERS =====
CGCCAG -> 5397
CTGGCG -> 5265
GCCAGC -> 4829
GCTGGC -> 4760
CCAGCG -> 4609
CGCTGG -> 4496
CCAGCA -> 4085
CAGCGC -> 3954
TGCTGG -> 3811
GCGGCG -> 3737
CGCCAG ***** (5397)
CTGGCG ***** (5265)
GCCAGC ***** (4829)
GCTGGC ***** (4760)
CCAGCG ***** (4609)
CGCTGG ***** (4496)
CCAGCA ***** (4085)
CAGCGC ***** (3954)
TGCTGG ***** (3811)
GCGGCG ***** (3737)

Results saved.
```

(a) Resultados de *Escherichia coli str. K-12 substr. MG1655*

```
usuario@usuario-L00-15IRH8:~/K-mears$ ./Kmears sequence3.fasta 6
Sequence length: 4857450
Using k = 6

===== TOP 10 K-MERS =====
CGCCAG -> 6736
CTGGCG -> 6624
GCGGCG -> 6394
CGCCGC -> 6389
CCAGCG -> 5743
CAGCGC -> 5702
GCCAGC -> 5628
GCTGGC -> 5467
CGCTGG -> 5411
GCGCTG -> 5323
CGCCAG ***** (6736)
CTGGCG ***** (6624)
GCGGCG ***** (6394)
CGCCGC ***** (6389)
CCAGCG ***** (5743)
CAGCGC ***** (5702)
GCCAGC ***** (5628)
GCTGGC ***** (5467)
CGCTGG ***** (5411)
GCGCTG ***** (5323)

Results saved.
```

(b) Resultados de *Salmonella enterica subsp. enterica serovar Typhimurium str. LT2*

Figura 1: Resultados en los genomas proporcionados con $k = 6$

Con $k = 11$

```
usuario@usuario-L00-15IRH8:~/K-mears$ ./Kmears sequence2.fasta 11
Sequence length: 4639675
Using k = 11

===== TOP 10 K-MERS =====
CGCATCCGGCA -> 123
TGCCGGATGCG -> 115
CGCCGATCCG -> 114
CGGATGCGGCG -> 102
CGCATCCGGC -> 101
GCCGATCCGG -> 99
GATAAGCGTT -> 98
ACGCCGATCC -> 97
GCCGGATGCG -> 97
TGCTGATGCG -> 96
CGCATCCGGCA ***** (123)
TGCCGGATGCG ***** (115)
CGCCGATCCG ***** (114)
CGGATGCGGCG ***** (102)
CGCATCCGGC ***** (101)
GCCGATCCGG ***** (99)
GATAAGCGTT ***** (98)
ACGCCGATCC ***** (97)
GCCGGATGCG ***** (97)
TGCTGATGCG ***** (96)

Results saved.
```

(a) Resultados de *Escherichia coli str. K-12 substr. MG1655*

```
usuario@usuario-L00-15IRH8:~/K-mears$ ./Kmears sequence3.fasta 11
Sequence length: 4857450
Using k = 11

===== TOP 10 K-MERS =====
CGCCATCCGGC -> 106
GCCATCCGGCA -> 93
GCCGATGCGC -> 92
GCGCCAGCGCC -> 83
GGCCGGATAAG -> 80
AGGCCGATAA -> 80
TGCCGGATGCG -> 80
CGCTGGCGCG -> 78
GCTGGCGCTGG -> 76
GCTGGCGCGG -> 76
CGCCATCCGGC ***** (106)
GCCATCCGGCA ***** (93)
GCCGATGCGC ***** (92)
GCGCCAGCGCC ***** (83)
GGCCGGATAAG ***** (80)
AGGCCGATAA ***** (80)
TGCCGGATGCG ***** (80)
CGCTGGCGCG ***** (78)
GCTGGCGCTGG ***** (76)
GCTGGCGCGG ***** (76)

Results saved.
```

(b) Resultados de *Salmonella enterica subsp. enterica serovar Typhimurium str. LT2*

Figura 2: Resultados en los genomas proporcionados con $k = 11$

Con $k = 21$

```
usuario@usuario-L0Q-151RH8:~/K-mears$ ./Kmears sequence2.fasta 21
Sequence length: 4639675
Using k = 21

===== TOP 10 K-MERS =====
ATAAGGCGTTACGCCGCATC -> 43
GATAAGGCGTTACGCCGCAT -> 43
AAGGCGTTACGCCGCATCCG -> 41
GGATAAGGCGTTACGCCGCA -> 39
TAAGGCGTTACGCCGCATCC -> 39
CGGATGCGGCGTGAACGCTT -> 39
GGCGTTACGCCGCATCCGG -> 38
AGGCGTTACGCCGCATCCGG -> 38
GATGCGGCGTGAACGCTTAT -> 38
GGATGCGGCGTGAACGCTT -> 38
ATAAGGCGTTACGCCGCATC ***** (43)
GATAAGGCGTTACGCCGCAT ***** (43)
AAGGCGTTACGCCGCATCCG ***** (41)
GGATAAGGCGTTACGCCGCA ***** (39)
TAAGGCGTTACGCCGCATCC ***** (39)
CGGATGCGGCGTGAACGCTT ***** (39)
GGCGTTACGCCGCATCCGG ***** (38)
AGGCGTTACGCCGCATCCGG ***** (38)
GATGCGGCGTGAACGCTTAT ***** (38)
GGATGCGGCGTGAACGCTT ***** (38)

Results saved.
```

(a) Resultados de Escherichia coli str. K-12 substr. MG1655

```
usuario@usuario-L0Q-151RH8:~/K-mears$ ./Kmears sequence3.fasta 21
Sequence length: 4857450
Using k = 21

===== TOP 10 K-MERS =====
TCCCCGCTGGCGGGGAACA -> 51
GTTTATCCCCGCTGGCGGG -> 51
TATCCCCGCTGGCGGGGAA -> 51
ATCCCCGCTGGCGGGGAAC -> 51
TTATCCCCGCTGGCGGGGA -> 51
TTTATCCCCGCTGGCGGGG -> 50
CCCCGCTGGCGGGGAACAC -> 50
GGTTTATCCCCGCTGGCGGG -> 50
CGGTTATCCCCGCTGGCGGG -> 49
CCCGCTGGCGGGGAACACG -> 19
TCCCCGCTGGCGGGGAACA ***** (51)
GTTTATCCCCGCTGGCGGG ***** (51)
TATCCCCGCTGGCGGGGAA ***** (51)
ATCCCCGCTGGCGGGGAAC ***** (51)
TTATCCCCGCTGGCGGGGA ***** (51)
TTTATCCCCGCTGGCGGGG ***** (50)
CCCCGCTGGCGGGGAACAC ***** (50)
GGTTTATCCCCGCTGGCGGG ***** (50)
CGGTTATCCCCGCTGGCGGG ***** (49)
CCCGCTGGCGGGGAACACG ***** (19)

Results saved.
```

(b) Resultados de Salmonella enterica subsp. enterica serovar Typhimurium str. LT2

Figura 3: Resultados en los genomas proporcionados con $k = 21$

2.2. Resultados de frecuencia y datos estadísticos

Los resultados estadísticos que hemos obtenido se almacenan en un archivo de texto donde especifica lo siguiente: el uso del k para cada genoma, número de **k-mers** generados, cantidad única de **k-mers**, el **k-mer** más frecuente, el **k-mer** con una frecuencia mínima y los 20 **k-mers** más frecuentes en toda la secuencia.

Con $k = 6$

```
kmer_statistics.txt
1 k value: 6
2 Total k-mers: 4639670
3 Unique k-mers: 4096
4
5 Most frequent k-mer:
6 CGCCAG -> 5397
7
8 Least frequent k-mer:
9 CCTAGG -> 16
10
11 ===== TOP 20 K-MERS =====
12 CGCCAG : 5397
13 CTGGCG : 5265
14 GCCAGC : 4829
15 GCTGGC : 4760
16 CCAGCG : 4609
17 CGCTGG : 4496
18 CCAGCA : 4085
19 CAGCGC : 3954
20 TGCTGG : 3811
21 GCGGCG : 3737
22 CGCCGC : 3719
23 CAGCAG : 3713
24 TCGCCA : 3691
25 GCGCTG : 3635
26 CTGGCA : 3613
27 ATCGCC : 3553
28 CCGCCA : 3529
29 TCCAGC : 3478
30 TGCCAG : 3474
31 CAGCAA : 3448
```

(a) Resultados estadísticos de Escherichia coli str. K-12 substr. MG1655

```
kmer_statistics.txt
1 k value: 6
2 Total k-mers: 4857445
3 Unique k-mers: 4096
4
5 Most frequent k-mer:
6 CGCCAG -> 6736
7
8 Least frequent k-mer:
9 CCTAGA -> 10
10
11 ===== TOP 20 K-MERS =====
12 CGCCAG : 6736
13 CTGGCG : 6624
14 GCGGCG : 6394
15 CGCCGC : 6389
16 CCAGCG : 5743
17 CAGCGC : 5702
18 GCCAGC : 5628
19 GCTGGC : 5467
20 CGCTGG : 5411
21 GCGCTG : 5323
22 GCGGCG : 5032
23 GCGGCG : 5021
24 CCGCGC : 4990
25 GCGGCC : 4968
26 GCGCCA : 4939
27 GCGCGG : 4855
28 GCGGCG : 4795
29 TGGCGC : 4787
30 CGCGCC : 4749
31 CGGCGG : 4449
```

(b) Resultados estadísticos de Salmonella enterica subsp. Typhimurium str. LT2

Figura 4: Resultados en los genomas proporcionados con $k = 6$

El espacio de estados teórico posible es $4^k = 4^6 = 4,096$.

■ Muestra 1:

- Total de k-mers: 4,639,670
- K-mers únicos: 4,096
- K-mer más frecuente: CGCCAG con una frecuencia de 5,397

■ Muestra 2:

- Total de k-mers: 4,857,445
- K-mers únicos: 4,096
- K-mer más frecuente: CGCCAG con una frecuencia de 6,736

Con $k = 11$

```
kmer_statistics.txt X
kmer_statistics.txt
1 k value: 11
2 Total k-mers: 4639665
3 Unique k-mers: 2196835
4
5 Most frequent k-mer:
6 CGCATCCGGCA -> 123
7
8 Least frequent k-mer:
9 GTTGGATTCTT -> 1
10
11 ===== TOP 20 K-MERS =====
12 CGCATCCGGCA : 123
13 TGCCGGATGCG : 115
14 CGCCGCATCCG : 114
15 CGGATGCGGCG : 102
16 CCGCATCCGGC : 101
17 GCCGCATCCGG : 99
18 GATAAGGCGTT : 98
19 ACGCCGCATCC : 97
20 GCCGGATGCGG : 97
21 TGCCTGATGCG : 96
22 CGGATAAGGCG : 96
23 CTTATCAGGCC : 93
24 GATGCGACGCT : 93
25 TTATCAGGCCT : 92
26 CCGGATGCGGC : 92
27 GGATAAGGCGT : 92
28 GGATGCGGCGT : 88
29 CGCCTTATCCG : 88
30 TATCAGGCCTA : 85
31 ATCAGGCCTAC : 84
```

(a) Resultados estadísticos de Escherichia coli str. K-12 substr. MG1655

```
kmer_statistics.txt X
kmer_statistics.txt
1 k value: 11
2 Total k-mers: 4857440
3 Unique k-mers: 2237344
4
5 Most frequent k-mer:
6 CGCCATCCGGC -> 106
7
8 Least frequent k-mer:
9 TATATGCTCGG -> 1
10
11 ===== TOP 20 K-MERS =====
12 CGCCATCCGGC : 106
13 GCCATCCGGCA : 93
14 GCCGGATGCGG : 92
15 GCGCCAGCGCC : 83
16 GGCCGGATAAG : 80
17 AGGCCGGATAA : 80
18 TGCCGGATGGC : 80
19 CCGCTGGCGCG : 78
20 GCTGGCGCTGG : 76
21 GCTGGCGCGGG : 76
22 CGCCGCCAGCG : 76
23 CCAGCGCCAGC : 75
24 CGCTGGCGCGG : 74
25 CAGCGCCAGCG : 74
26 CGCCAGCGCCG : 71
27 CGCCAGCGGCG : 71
28 CGCTGGCGGCG : 71
29 GTAGCCGGAT : 70
30 CCCGCTGGCGC : 69
31 TGGCGCTGGCG : 68
```

(b) Resultados estadísticos de Salmonella enterica subsp. Typhimurium str. LT2

Figura 5: Resultados en los genomas proporcionados con $k = 11$

El espacio de estados teórico es $4^{11} = 4,194,304$.

■ Muestra 1:

- Total de k-mers: 4,639,665
- K-mers únicos: 2,196,835
- K-mer más frecuente: CGCATCCGGCA (123 apariciones)

■ Muestra 2:

- Total de k-mers: 4,857,440
- K-mers únicos: 2,237,344
- K-mer más frecuente: CGCCATCCGGC (106 apariciones)

Con $k = 21$

```
kmer_statistics.txt X
kmer_statistics.txt
1 k value: 21
2 Total k-mers: 4639655
3 Unique k-mers: 4562500
4
5 Most frequent k-mer:
6 ATAAGGCGTTCACGCCGCATC -> 43
7
8 Least frequent k-mer:
9 GGAGGGTCAATTGCTGACTTT -> 1
10
11 ===== TOP 20 K-MERS =====
12 ATAAGGCGTTCACGCCGCATC : 43
13 GATAAGGCGTTCACGCCGCAT : 43
14 AAGGCGTTCACGCCGCATCCG : 41
15 GGATAAGGCGTTCACGCCGCA : 39
16 TAAGGCGTTCACGCCGCATCC : 39
17 CGGATGCGCGCTGAACGCCTT : 39
18 GCGGTTACGCCGCATCCGCG : 38
19 AGGCGTTACGCCGCATCCGCG : 38
20 GATGCGCGCTGAACGCCTTAT : 38
21 GGATGCGCGCTGAACGCCTTA : 38
22 TGCGGCGTGAACGCCTTATCC : 36
23 ATGCGGCGTGAACGCCTTATC : 36
24 GCGTTACGCCGCATCCGCGCA : 35
25 CGGATAAGGCGTTCACGCCGC : 35
26 GCCGATGCGCGCTGAACGCC : 35
27 GCGGCGTGAACGCCTTATCCG : 34
28 GATAAGGCGTTACGCCGCAT : 34
29 ATAAGGCGTTACGCCGCATC : 33
30 CCGGATGCGCGCTGAACGCCT : 33
31 TGCCGGATGCGCGCTGAACGC : 33
```

(a) Resultados estadísticos de *Escherichia coli* str. K-12 substr. MG1655

```
kmer_statistics.txt X
kmer_statistics.txt
1 k value: 21
2 Total k-mers: 4857430
3 Unique k-mers: 4807695
4
5 Most frequent k-mer:
6 TCCCCGCTGGCGCGGGGAACA -> 51
7
8 Least frequent k-mer:
9 GTTATAGGGTTTATGGTTATT -> 1
10
11 ===== TOP 20 K-MERS =====
12 TCCCCGCTGGCGCGGGGAACA : 51
13 GTTATCCCCGCTGGCGCGGG : 51
14 TATCCCCGCTGGCGCGGGGAA : 51
15 ATCCCCGCTGGCGCGGGGAAC : 51
16 TTATCCCCGCTGGCGCGGGGA : 51
17 TTTATCCCCGCTGGCGCGGGG : 50
18 CCCCCTGGCGCGGGGAACAC : 50
19 GGTATATCCCCGCTGGCGCGG : 50
20 CGTTTATCCCCGCTGGCGCG : 49
21 CCGCTGGCGCGGGGAACACG : 19
22 GCGGTTATCCCCGCTGGCGG : 16
23 CCGCTGGCGCGGGGAACACT : 15
24 GCTGATGGCGCTGGCGCTTAT : 14
25 CCTGATGGCGCTGGCGCTTAT : 14
26 ACGTTTATCCCCGCTGGCGG : 14
27 CGTGCGCTTATCAGGCCTAC : 14
28 TGCGCTGGCGTTATCAGGCC : 13
29 ATGGCGCTGGCGTTATCAGGC : 13
30 GCGCTGGCGTTATCAGGCCTA : 13
31 GATGGCGCTGGCGTTATCAGG : 12
```

(b) Resultados estadísticos de *Salmonella enterica* subsp. Typhimurium str. LT2

Figura 6: Resultados en los genomas proporcionados con $k = 21$

El espacio de estados teórico es $4^{21} \approx 4,39 \times 10^{12}$.

■ Muestra 1:

- Total de k-mers: 4,639,655
- K-mers únicos: 4,562,500
- K-mer más frecuente: ATAAGGCGTTCACGCCGCATC (43 apariciones)

■ Muestra 2:

- Total de k-mers: 4,857,430
- K-mers únicos: 4,807,695
- K-mer más frecuente: TCCCCGCTGGCGCGGGGAACA (51 apariciones)

Tabla comparativa

Cuadro 1: Evolución de la unicidad de k-mers

Valor de k	Total k-mers (promedio)	K-mers Únicos (promedio)	Proporción Únicos
6	$4,74 \times 10^6$	$4,09 \times 10^3$	$\approx 0,08 \%$
11	$4,74 \times 10^6$	$2,21 \times 10^6$	$\approx 46,6 \%$
21	$4,74 \times 10^6$	$4,68 \times 10^6$	$\approx 98,7 \%$

2.3. Comparación con dos genomas

Ahora que hemos probado el **k-mers** con un solo genoma, debemos poder usar este mismo procesamiento para la comparación de dos genomas distintos, con el objetivo de poder encontrar similitudes y las diferencias en cada composición genética. Para esta comparación usaremos los genomas proporcionados: *Escherichia coli* str. K-12 substr. MG1655 y *Salmonella enterica* subsp. enterica serovar Typhimurium str. LT2.

1. Debemos construir conjuntos de **k-mers** únicos con estructuras eficientes que ayuden a optimizar búsquedas y comparaciones.
2. Obtenemos los conjuntos de **k-mers** que corresponden a cada genoma.
3. Una vez tenemos los conjuntos se va calculando la unión e intersección de conjuntos, representando la cantidad total de **k-mers** y representando la cantidad de subsecuencias comunes entre ambos organismos.
4. Para medir la similitud genómica se utiliza el índice de Jaccard usando la relación entre intersección y unión de los conjuntos.

Entonces en esta parte debemos poder procesar ambas secuencias de los FASTA para realizar la comparación.

```
1 using KmerSet = unordered_set<string>;
2
3 KmerSet buildKmerSet(const string& sequence, int k) {
4     KmerSet kmers;
5     kmers.reserve(sequence.size());
6
7     for (size_t i = 0; i + k <= sequence.size(); ++i) {
8         kmers.insert(sequence.substr(i, k));
9     }
10
11     return kmers;
12 }
13
14 void sharedKmers(const KmerSet& set1, const KmerSet& set2, int limit = 10) {
15     int count = 0;
16
17     for (const auto& kmer : set1) {
18         if (set2.count(kmer)) {
19             cout << "Shared kmer: " << kmer << endl;
20             if (++count >= limit) {
21                 break;
22             }
23         }
24     }
25
26     cout << "Shared kmers: " << count << endl;
27 }
28
29 void uniqueKmers(const KmerSet& set1, const KmerSet& set2, const string& label, int
    limit = 10) {
30     int count = 0;
31
32     for (const auto& kmer : set1) {
33         if (!set2.count(kmer)) {
34             cout << "Unique kmer in " << label << ": " << kmer << endl;
35             if (++count >= limit) {
36                 break;
37             }
38         }
39     }
40 }
41
42 void compareKmers(const KmerSet& set1, const KmerSet& set2) {
43     size_t intersection = 0;
44
45     for (const auto& kmer : set1) {
46         if (set2.count(kmer)) {
47             intersection++;
48         }
49     }
```

```

49     }
50
51     size_t union_size = set1.size() + set2.size() - intersection;
52     double jaccard_index = (double)intersection / union_size;
53
54     cout << "\n===== COMPARISON =====\n";
55
56     cout << "Set 1 size: " << set1.size() << endl;
57     cout << "Set 2 size: " << set2.size() << endl;
58     cout << "Shared k-mers: " << intersection << endl;
59     cout << "Unique in Seq1: " << (set1.size() - intersection) << endl;
60     cout << "Unique in Seq2: " << (set2.size() - intersection) << endl;
61     cout << "Union: " << union_size << endl;
62
63     cout << "\nJaccard Index: " << jaccard_index << endl;
64
65     if (jaccard_index > 0.8) {
66         cout << "Interpretation: Very high similarity (almost identical genomes)\n"
67             << endl;
68     }
69     else if (jaccard_index > 0.5) {
70         cout << "Interpretation: Moderate similarity (closely related organisms)\n"
71             << endl;
72     }
73     else if (jaccard_index > 0.2) {
74         cout << "Interpretation: Low similarity (related but different genomes)\n";
75     }
76     else {
77         cout << "Interpretation: Very low similarity (distant organisms)\n";
78     }
79
80     sharedKmers(set1, set2, 10);
81     uniqueKmers(set1, set2, "Sequence 1", 10);
82     uniqueKmers(set2, set1, "Sequence 2", 10);
83 }
84
85 void statistics(const string& filename, const KmerMap& kmers, const vector<pair<
86 string, int>> sorted, int k, size_t total_kmers) {
87     ofstream outputfile(filename);
88
89     if (!outputfile.is_open()) {
90         cerr << "Error opening output file: " << filename << endl;
91         return;
92     }
93
94     outputfile << "k value: " << k << "\n";
95     outputfile << "Total k-mers: " << total_kmers << "\n";
96     outputfile << "Unique k-mers: " << kmers.size() << "\n\n";
97
98     outputfile << "Most frequent k-mer:\n";
99     outputfile << sorted.front().first << " -> " << sorted.front().second << "\n\n"
100         << endl;
101
102     outputfile << "Least frequent k-mer:\n";
103     outputfile << sorted.back().first << " -> " << sorted.back().second << "\n\n";
104
105     outputfile << "==== TOP 20 K-MERS ==== \n";
106
107     for (int i = 0; i < 20 && i < sorted.size(); ++i) {
108         outputfile << sorted[i].first << " : " << sorted[i].second << "\n";
109     }
110
111     outputfile.close();

```


108 }

Entonces vamos a proceder a ejecutar los genomas correspondientes con valores distintos en k .

Con $k = 6$

Este nivel mide la composición básica de hexámeros.

- **Índice de Jaccard:** 1,000
- **Interpretación:** Saturación total. Ambas secuencias comparten la totalidad de las 4,096 combinaciones posibles. Se interpreta como una similitud muy alta, aunque de baja especificidad biológica.

```
usuario@usuario-LQ-15IRH8:~/K-mers$ ./Kmers sequence2.fasta sequence3.fasta 6
Seq1 length: 4639675
Seq2 length: 4857450
Using k = 6

----- COMPARISON -----
Set 1 size: 4096
Set 2 size: 4096
Shared k-mers: 4096
Unique in Seq1: 0
Unique in Seq2: 0
Union: 4096

Jaccard Index: 1
Interpretation: Very high similarity (almost identical genomes)
Shared kmer: CTAGCA
Shared kmer: GTCTAG
Shared kmer: ACTAGA
Shared kmer: CTAGCT
Shared kmer: GACTAG
Shared kmer: TCCTAG
Shared kmer: TCTAGC
Shared kmer: CCTAGC
Shared kmer: TACTAG
Shared kmer: CCTAGT
Shared kmers: 10
```

Figura 7: Análisis de comparación con los genomas

Con $k = 11$

Representa un nivel intermedio con mayor especificidad.

- **K-mers compartidos:** 1,502,853
- **Unión (Total):** 2,931,326
- **Índice de Jaccard:** 0,512687
- **Interpretación:** Similitud moderada. Los genomas presentan bloques estructurales comunes, sugiriendo que son organismos estrechamente relacionados.

```
usuario@usuario-LQ-15IRH8:~/K-mers$ ./Kmers sequence2.fasta sequence3.fasta 11
Seq1 length: 4639675
Seq2 length: 4857450
Using k = 11

----- COMPARISON -----
Set 1 size: 2196835
Set 2 size: 2237344
Shared k-mers: 1502853
Unique in Seq1: 693982
Unique in Seq2: 734491
Union: 2931326

Jaccard Index: 0.512687
Interpretation: Moderate similarity (closely related organisms)
Shared kmer: AACGCCTTACT
Shared kmer: CAATGTTGAC
Shared kmer: GCATGTTGCA
Shared kmer: CGACACACGG
Shared kmer: ACAGACACGG
Shared kmer: ACAGACACGG
Shared kmer: GCACACACGA
Shared kmer: ACAACGCTGG
Shared kmer: GGTACACAA
Shared kmer: GGTGACTGT
Shared kmers: 10
Unique kmer in Sequence 1: TAGTAAGTATT
Unique kmer in Sequence 1: TTAGTAAGTAT
Unique kmer in Sequence 1: GCCTTACTAGG
Unique kmer in Sequence 1: GAGCACCGCA
Unique kmer in Sequence 1: TTAGAGCAAC
Unique kmer in Sequence 1: GCTTTAGAGC
Unique kmer in Sequence 1: CAGTCACATA
Unique kmer in Sequence 1: CGAACGACCA
Unique kmer in Sequence 1: GCGACGAGCC
Unique kmer in Sequence 2: AAAAACTAGCA
Unique kmer in Sequence 2: AAAAAACTA
Unique kmer in Sequence 2: GTCCATGATA
Unique kmer in Sequence 2: ACAGCGAGATA
Unique kmer in Sequence 2: GCGACAGCAG
Unique kmer in Sequence 2: GCGCCATCACC
Unique kmer in Sequence 2: GCGCCGATGA
Unique kmer in Sequence 2: AGCGCGCCAT
Unique kmer in Sequence 2: CAGCGCGCCA
Unique kmer in Sequence 2: GAGCGCGCCG
```

Figura 8: Análisis de comparación con los genomas

Con $k = 21$

Análisis de alta precisión sobre la estructura genómica.

- **K-mers compartidos:** 147,642
- **Unión (Total):** 9,222,553
- **Índice de Jaccard:** 0,0160088
- **Interpretación:** Similitud muy baja. La gran divergencia indica que, a nivel de nucleótidos de longitud 21, los genomas son fundamentalmente diferentes, clasificándose como organismos distantes.

```

usuario@usuario-LQ-15IRH8:~/Kmers $ ./Kmers sequence2.fasta sequence3.fasta 21
Seq1 length: 4639675
Seq2 length: 4857450
Using k = 21

----- COMPARISON -----
Set 1 size: 4562500
Set 2 size: 4807695
Shared k-mers: 147642
Unique in Seq1: 4414858
Unique in Seq2: 4660053
Union: 9222553

Jaccard Index: 0.0160088
Interpretation: Very low similarity (distant organisms)
Shared kmer: CAGTTTGTGAAATCATCGTA
Shared kmer: TAACAGTTTGTGAAATCATC
Shared kmer: TTAACAGTTTGTGAAATCAT
Shared kmer: TAGCAGCTTAAACAGTTTGATG
Shared kmer: TGTAGCAGCTTAAACAGTTTGA
Shared kmer: AATTGTAGCAGCTTAAACAGTT
Shared kmer: TCAAGTTCAATTGTAGCAGCT
Shared kmer: TATCAAGTTCAATTGTAGCAC
Shared kmer: TGACATATATCAAGTTCAATT
Shared kmer: TCGTTGACATATATCAAGTTTC
Shared kmers: 10
Unique kmer in Sequence 1: AAAAAACGCTTAGTAAGTAT
Unique kmer in Sequence 1: AATAAAAAACGCTTAGTAAG
Unique kmer in Sequence 1: CCAATTAATAAACGCTTAGT
Unique kmer in Sequence 1: CACCAATAAAAAACGCTTA
Unique kmer in Sequence 1: TCACCAATAAAAAACGCTT
Unique kmer in Sequence 1: ATCACCATAAAAAACGCTT
Unique kmer in Sequence 1: TATCACCATAAAAAACGCC
Unique kmer in Sequence 1: ATATCACCATAAAAAACGCC
Unique kmer in Sequence 1: AATATCACCATAAAAAACG
Unique kmer in Sequence 1: AAAATACCAATAAAAAA
Unique kmer in Sequence 2: GCTGTATTTTTTAAATAA
Unique kmer in Sequence 2: ACCTGCTGTATTTTTAAAT
Unique kmer in Sequence 2: ATAACGTGCTGTATTTTTAA
Unique kmer in Sequence 2: AATAACGTGCTGTATTTTTA
Unique kmer in Sequence 2: AAAATAACGTGCTGTATTTT
Unique kmer in Sequence 2: CAAAAAATACGTGCTGTATTT
Unique kmer in Sequence 2: ACAAAATAACGTGCTGTATTT
Unique kmer in Sequence 2: AAAAAATAACGTGCTGTATTT
Unique kmer in Sequence 2: AAAAACTAACAAATAACGTG
Unique kmer in Sequence 2: AAAAACTAACAAATAACGTG

```

Figura 9: Análisis de comparación con los genomas

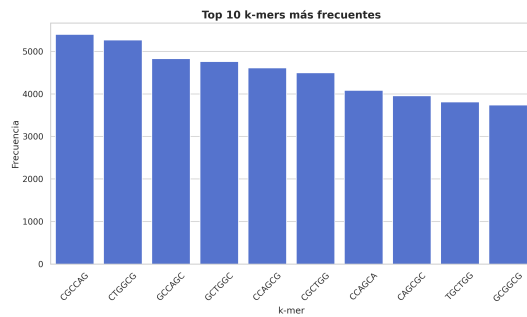
Tabla comparativa

Cuadro 2: Resumen de similitud Jaccard por ventana k

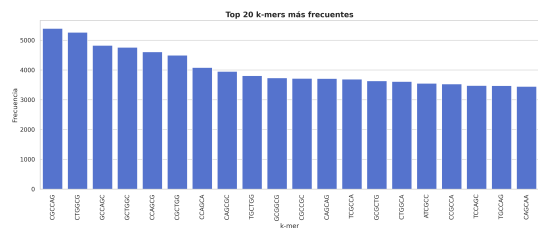
Ventana (k)	Índice de Jaccard	Clasificación Evolutiva
6	1,000	Relacionados moderadamente Distantes
11	0,512	
21	0,016	

2.4. Visualización de resultados

Por último pasamos a la visualización de los resultados que se obtuvieron en el análisis estadístico. Los gráficos presentados pertenecen al **Genoma Escherichia coli str. K-12 substr. MG1655** con un $k = 6$.



(a) Top 10 más repetidos



(b) Top 20 más repetidos

Figura 10: Resultados en gráficos de barras

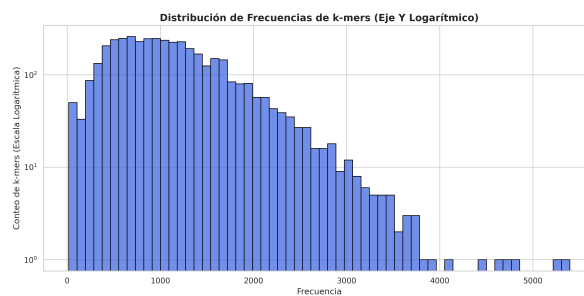


Figura 11: Histograma resultante



Figura 12: Nube de Palabras resultante

3. Programa Completo

El programa completo seria el siguiente:

Código C++

```
1 #include <omp.h>
2 #include <iostream>
3 #include <vector>
4 #include <cmath>
5 #include <fstream>
6 #include <string>
7 #include <unordered_map>
8 #include <unordered_set>
9 #include <algorithm>
10
11 using namespace std;
```

```
12 using KmerMap = unordered_map<string, int>;
13 using KmerSet = unordered_set<string>;
14
15 struct FastaSequence {
16     string header;
17     string sequence;
18 };
19
20 vector<FastaSequence> readFastaFile(const string& filename) {
21     ifstream file_fasta(filename);
22     vector<FastaSequence> sequences;
23
24     if (!file_fasta.is_open()) {
25         cerr << "Error opening file: " << filename << endl;
26         return sequences;
27     }
28
29     string line;
30     FastaSequence current_sequence;
31
32     while (getline(file_fasta, line)) {
33         if (line.empty()) continue;
34
35         if (line[0] == '>') {
36             if (!current_sequence.header.empty()) {
37                 sequences.push_back(current_sequence);
38                 current_sequence = FastaSequence();
39             }
40             current_sequence.header = line.substr(1);
41         }
42         else {
43             current_sequence.sequence += line;
44         }
45     }
46
47     if (!current_sequence.header.empty()) {
48         sequences.push_back(current_sequence);
49     }
50
51     return sequences;
52 }
53
54 KmerMap countMers(string sequence, int k) {
55     int n_threads = omp_get_max_threads();
56     vector<KmerMap> local_kmer_maps(n_threads);
57
58     #pragma omp parallel
59     {
60         int tid = omp_get_thread_num();
61         auto& local_map = local_kmer_maps[tid];
62
63         local_map.reserve(100000);
64
65         #pragma omp for schedule(dynamic, 10000)
66         for (size_t i = 0; i <= sequence.size() - k; ++i) {
67             string kmer = sequence.substr(i, k);
68             local_map[kmer]++;
69         }
70     }
71
72     KmerMap global;
73     for (auto& local : local_kmer_maps) {
74         for (const auto& [kmer, freq] : local) {
```

```

75         global[kmer] += freq;
76     }
77 }
78
79 return global;
80 }
81
82 KmerSet buildKmerSet(const string& sequence, int k) {
83     KmerSet kmers;
84     kmers.reserve(sequence.size());
85
86     for (size_t i = 0; i + k <= sequence.size(); ++i) {
87         kmers.insert(sequence.substr(i, k));
88     }
89
90     return kmers;
91 }
92
93 void sharedKmers(const KmerSet& set1, const KmerSet& set2, int limit = 10) {
94     int count = 0;
95
96     for (const auto& kmer : set1) {
97         if (set2.count(kmer)) {
98             cout << "Shared kmer: " << kmer << endl;
99             if (++count >= limit) {
100                 break;
101             }
102         }
103     }
104
105     cout << "Shared kmers: " << count << endl;
106 }
107
108 void uniqueKmers(const KmerSet& set1, const KmerSet& set2, const string& label, int
    limit = 10) {
109     int count = 0;
110
111     for (const auto& kmer : set1) {
112         if (!set2.count(kmer)) {
113             cout << "Unique kmer in " << label << ": " << kmer << endl;
114             if (++count >= limit) {
115                 break;
116             }
117         }
118     }
119 }
120
121 void compareKmers(const KmerSet& set1, const KmerSet& set2) {
122     size_t intersection = 0;
123
124     for (const auto& kmer : set1) {
125         if (set2.count(kmer)) {
126             intersection++;
127         }
128     }
129
130     size_t union_size = set1.size() + set2.size() - intersection;
131     double jaccard_index = (double)intersection / union_size;
132
133     cout << "\n----- COMPARISON ----- \n";
134
135     cout << "Set 1 size: " << set1.size() << endl;
136     cout << "Set 2 size: " << set2.size() << endl;

```

```

137     cout << "Shared k-mers: " << intersection << endl;
138     cout << "Unique in Seq1: " << (set1.size() - intersection) << endl;
139     cout << "Unique in Seq2: " << (set2.size() - intersection) << endl;
140     cout << "Union: " << union_size << endl;
141
142     cout << "\nJaccard Index: " << jaccard_index << endl;
143
144     if (jaccard_index > 0.8) {
145         cout << "Interpretation: Very high similarity (almost identical genomes)\n"
146             ;
147     }
148     else if (jaccard_index > 0.5) {
149         cout << "Interpretation: Moderate similarity (closely related organisms)\n"
150             ;
151     }
152     else if (jaccard_index > 0.2) {
153         cout << "Interpretation: Low similarity (related but different genomes)\n";
154     }
155     else {
156         cout << "Interpretation: Very low similarity (distant organisms)\n";
157     }
158
159     sharedKmers(set1, set2, 10);
160     uniqueKmers(set1, set2, "Sequence 1", 10);
161     uniqueKmers(set2, set1, "Sequence 2", 10);
162 }
163
164 vector<pair<string, int>> sortKmers(const unordered_map<string, int>& kmers) {
165     vector<pair<string, int>> vec(kmers.begin(), kmers.end());
166
167     sort(vec.begin(), vec.end(), [](const auto& a, const auto& b) {
168         return a.second > b.second;
169     });
170
171     return vec;
172 }
173
174 void statistics(const string& filename, const KmerMap& kmers, const vector<pair<
175 string, int>> sorted, int k, size_t total_kmers) {
176     ofstream outputfile(filename);
177
178     if (!outputfile.is_open()) {
179         cerr << "Error opening output file: " << filename << endl;
180         return;
181     }
182
183     outputfile << "k value: " << k << "\n";
184     outputfile << "Total k-mers: " << total_kmers << "\n";
185     outputfile << "Unique k-mers: " << kmers.size() << "\n\n";
186
187     outputfile << "Most frequent k-mer:\n";
188     outputfile << sorted.front().first << " -> " << sorted.front().second << "\n\n"
189         ;
190
191     outputfile << "Least frequent k-mer:\n";
192     outputfile << sorted.back().first << " -> " << sorted.back().second << "\n\n";
193
194     outputfile << "==== TOP 20 K-MERS =====\n";
195
196     for (int i = 0; i < 20 && i < sorted.size(); ++i) {
197         outputfile << sorted[i].first << " : " << sorted[i].second << "\n";
198     }
199 }

```

```
196     outputfile.close();
197 }
198
199 void csvResults(const string& filename, const vector<pair<string, int>>& sorted) {
200     ofstream outputfile(filename);
201
202     if (!outputfile.is_open()) {
203         cerr << "Error opening output file: " << filename << endl;
204         return;
205     }
206
207     outputfile << "k-mer,Frequency\n";
208
209     for (const auto& [kmer, freq] : sorted) {
210         outputfile << kmer << "," << freq << "\n";
211     }
212
213     outputfile.close();
214 }
215
216 void worldCloud(const vector<pair<string, int>>& sorted, int top = 10) {
217     int max_freq = sorted.front().second;
218
219     for (int i = 0; i < top && i < sorted.size(); ++i) {
220         int stars = (50 * sorted[i].second) / max_freq;
221         cout << sorted[i].first << " ";
222         for (int j = 0; j < stars; ++j) {
223             cout << "*";
224         }
225         cout << " (" << sorted[i].second << ")";
226         cout << endl;
227     }
228 }
229
230 int main(int argc, char* argv[]) {
231     if (argc == 3) {
232         string file = "Files/" + string(argv[1]);
233         int k = stoi(argv[2]);
234
235         auto seqs = readFastaFile(file);
236
237         if (seqs.empty()) {
238             cerr << "Error reading FASTA file\n";
239             return 1;
240         }
241
242         string seq = seqs[0].sequence;
243
244         cout << "Sequence length: " << seq.size() << endl;
245         cout << "Using k = " << k << endl;
246
247         auto kmers = countMers(seq, k);
248         auto sorted = sortKmers(kmers);
249
250         cout << "\n----- TOP 10 K-MERS ----- \n";
251
252         for (int i = 0; i < 10 && i < sorted.size(); ++i) {
253             cout << sorted[i].first << " -> " << sorted[i].second << endl;
254         }
255
256         worldCloud(sorted, 10);
257
258         size_t total_kmers = seq.size() - k + 1;
```

```

259     statistics("kmer_statistics.txt", kmers, sorted, k, total_kmers);
260     csvResults("kmer_frequencies.csv", sorted);
261
262     cout << "\nResults saved.\n";
263 }
264 else if (argc == 4) {
265     string file1 = "Files/" + string(argv[1]);
266     string file2 = "Files/" + string(argv[2]);
267     int k = stoi(argv[3]);
268
269     auto seqs1 = readFastaFile(file1);
270     auto seqs2 = readFastaFile(file2);
271
272     if (seqs1.empty() || seqs2.empty()) {
273         cerr << "Error reading FASTA files\n";
274         return 1;
275     }
276
277     string seq1 = seqs1[0].sequence;
278     string seq2 = seqs2[0].sequence;
279
280     cout << "Seq1 length: " << seq1.size() << endl;
281     cout << "Seq2 length: " << seq2.size() << endl;
282     cout << "Using k = " << k << endl;
283
284     auto set1 = buildKmerSet(seq1, k);
285     auto set2 = buildKmerSet(seq2, k);
286
287     compareKmears(set1, set2);
288 }
289 else {
290     cerr << "Usage:\n";
291     cerr << "Single sequence:\n";
292     cerr << "./Kmears <fasta> <k>\n\n";
293     cerr << "Genome comparison:\n";
294     cerr << "./Kmears <fasta1> <fasta2> <k>\n";
295     return 1;
296 }
297
298     return 0;
299 }

```

Código Python

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4 from wordcloud import WordCloud
5
6 csv_file = "kmer_frequencies.csv"
7 df = pd.read_csv(csv_file)
8
9 sns.set_theme(style="whitegrid")
10
11 sorted_df = df.sort_values(by="Frequency", ascending=False)
12
13 single_color = "royalblue"
14
15 plt.figure(figsize=(10, 6))
16 top10 = sorted_df.head(10)
17 sns.barplot(x="k-mer", y="Frequency", data=top10, color=single_color)
18 plt.title("Top 10 k-mers mas frecuentes", fontsize=14, fontweight='bold')

```



```

19 plt.xlabel("k-mer", fontsize=12)
20 plt.ylabel("Frecuencia", fontsize=12)
21 plt.xticks(rotation=45, fontsize=11)
22 plt.tight_layout()
23 plt.savefig("top10_kmers.png", dpi=300)
24 plt.show()
25
26 plt.figure(figsize=(14, 6))
27 top20 = sorted_df.head(20)
28 sns.barplot(x="k-mer", y="Frequency", data=top20, color=single_color)
29 plt.title("Top 20 k-mers mas frecuentes", fontsize=14, fontweight='bold')
30 plt.xlabel("k-mer", fontsize=12)
31 plt.ylabel("Frecuencia", fontsize=12)
32 plt.xticks(rotation=90, fontsize=11)
33 plt.tight_layout()
34 plt.savefig("top20_kmers.png", dpi=300)
35 plt.show()
36
37 plt.figure(figsize=(12, 6))
38 sns.histplot(data=df, x="Frequency", bins=60, color=single_color, edgecolor="black"
39 )
40 plt.yscale("log")
41 plt.title("Distribucion de Frecuencias de k-mers (Eje Y Logaritmico)", fontsize=14,
42 fontweight='bold')
43 plt.xlabel("Frecuencia", fontsize=12)
44 plt.ylabel("Conteo de k-mers (Escala Logaritmica)", fontsize=12)
45 plt.tight_layout()
46 plt.savefig("kmer_histogram.png", dpi=300)
47 plt.show()
48
49 word_freq = dict(zip(df["k-mer"], df["Frequency"]))
50
51 wordcloud = WordCloud(
52     width=1200,
53     height=600,
54     background_color='white',
55     colormap='plasma'
56 ).generate_from_frequencies(word_freq)
57
58 plt.figure(figsize=(14, 7))
59 plt.imshow(wordcloud, interpolation='bilinear')
60 plt.axis("off")
61 plt.title("Nube de Palabras Genetica", fontsize=16, fontweight='bold')
62 plt.tight_layout()
63 plt.savefig("kmer_wordcloud.png", dpi=300)
64 plt.show()
65
66 print("\n==== ESTADISTICAS BASICAS =====")
67 print(f"Total de k-mers unicos: {len(df)}")
68 print(f"Frecuencia maxima: {df['Frequency'].max()}")
69 print(f"Frecuencia minima: {df['Frequency'].min()}")
70 print(f"Frecuencia promedio: {df['Frequency'].mean():.2f}")
71 print(f"Frecuencia mediana: {df['Frequency'].median():.2f}")
72
73 print("\nArchivos generados:")
74 print("- top10_kmers.png")
75 print("- top20_kmers.png")
76 print("- kmer_histogram.png")
77 print("- kmer_wordcloud.png")

```